

Design_Rationale_REQ3

A. NPC Implementation

Design 1: Create NPC-specific classes that inherit directly from Actor.

Pros	Cons
Each NPC can implement logic specific to its needs without constraints from a base class.	Common logic must be re-implemented in each NPC class.
NPCs with minimal or no monologue logic can avoid inheriting unnecessary functionality.	Adding new shared features would require modifying every NPC class instead of a single base class.
The class hierarchy is flattened, reducing structural complexity.	The lack of a shared abstraction for conversational NPCs makes the system less modular and harder to extend.

Design 2: Create an abstract class with shared logic for NPCs, which the NPCs inherit from.

Pros	Cons
The abstract <code>ConversationalActor</code> class makes it easy to create new NPCs by reusing shared features.	Similar NPC behaviors need to be updated individually, which is time-consuming and error-prone.
Monologues can be added with or without conditions, supporting context-sensitive dialogue.	New NPCs add only the behaviors they need, but shared behaviors may lead to duplicated code, increasing maintenance effort.
Changes to the listening logic only need to be made in one place, simplifying debugging and updates.	If some NPCs need specialized listening logic, the base class may become too generic to meet their needs.
The abstract <code>initMonologues()</code> method provides a clear extension point for adding NPC-specific dialogue.	
Each NPC class focuses on its monologues and behaviors, while the base class manages the mechanics.	

B. Monologue Implementation

Design 1: Separate monologue as its own class

Pros	Cons
Monologue logic is encapsulated in its own class, making the <code>ConversationalActor</code> class simpler and more focused.	Adding a fixed line requires creating a new Monologue and updating maps, while embedding a constant in the Actor is simpler.
The <code>Monologue</code> can be reused across different actor types or even non-actor entities.	
Monologue-related logic is isolated, reducing the risk of unintended interactions with other actor behaviors.	

Design 2: Embed monologue in conversational actor class

Pros	Cons
All monologue-related logic is centralized within the <code>ConversationalActor</code> , reducing the need for additional classes.	Monologue logic is tightly coupled with the actor, making it harder to reuse or extend for other types of actors.
Reduces the number of classes and interactions, simplifying the overall system.	If other actors need monologues, the logic must be duplicated or refactored.
	As monologues and conditions increase, the <code>ConversationalActor</code> class may become bloated and harder to maintain.

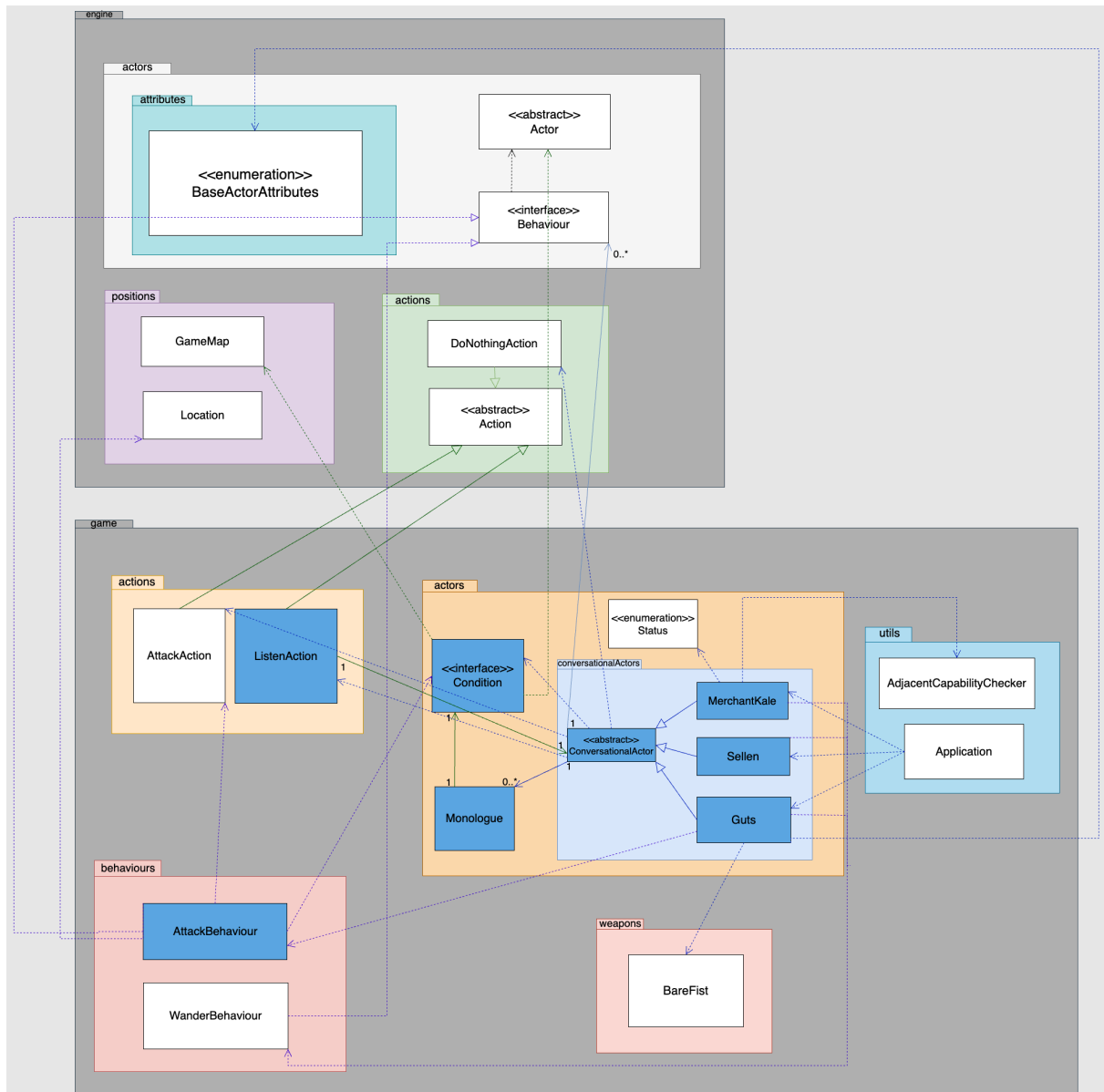
C. Condition Implementation

Design 1: Interface-based condition

Pros	Cons
Implemented as a functional interface for concise lambda-based instantiation.	Harder to enumerate all possible condition types at a glance.
Supports flexible condition building through composition over inheritance.	
Conditions are initialized in constructors, avoiding tight coupling to concrete implementations.	

Design 2: Abstract class-based condition

Pros	Cons
Allows defining common abstract methods and fields, reducing code duplication	Adding new conditions means creating new classes (can cause class explosion)
	Overhead for simple conditions



The diagram above shows the final design of requirement 3, which utilizes Design 2 for npc implementation, Design 1 for monologue and condition implementation.

Each class adheres strictly to the **Single Responsibility Principle**, with `ConversationalActor` handling only dialogue flow and monologue selection, `Monologue` encapsulating its text and the trigger `Condition`, `ListenAction` displaying speech, NPC subclasses like `Sellen`, `MerchantKale` and `Guts` defining only their own names, stats, behaviors and monologues, and `AttackBehaviour` scanning nearby locations for enemies and when one is found creating an `AttackAction` using the actor's weapon.

The **Open/Closed Principle (OCP)** is honored by keeping core classes closed to modification while allowing extensions via abstractions. The `Condition` interface is designed as a functional interface, enabling developers to define new conditions concisely using lambda expressions or method references. Adding a new `Behaviour` subclass requires no edits to existing code, just register it in the appropriate map. NPC types can introduce novel dialogue sequences in their own `initMonologues()` overrides without touching the base `ConversationalActor`. This approach keeps the framework stable even as new conversational features or NPC personalities are added.

All subclasses adhere to the **Liskov Substitution Principle (LSP)**, so any `ConversationalActor` reference in client code seamlessly accepts instances of `Sellen`, `MerchantKale`, or `Guts`. Each override of `initMonologues()` and every custom behavior respects the contracts defined in the abstract base, ensuring no unexpected side-effects or broken interactions when substituting one NPC type for another.

The **Interface Segregation Principle (ISP)** is reflected in narrowly scoped interfaces. The `Condition` interface exposes only a single method, avoiding "fat" interfaces that would force classes to implement unused methods. This lightweight interface design reduces coupling and enhances clarity.

Through **Dependency Inversion Principle (DIP)**, high-level modules like `ConversationalActor` rely on the abstract `Condition` and `Behaviour` interfaces, rather than concrete implementations. Low-level classes implement these interfaces, providing specific logic for monologue eligibility or NPC actions. This inversion promotes flexibility new condition types or behaviors can be introduced without altering high-level conversation logic.

Duplication is reduced by adhering to the **Don't Repeat Yourself (DRY)** principle through inheritance. Shared logic is in `ConversationalActor`, while NPC-specific data (monologues and conditions) is provided by subclasses. The design balances inheritance for NPC hierarchy with composition using maps and lists to manage monologues and behaviors. Monologues use Conditions for flexible, context-aware dialogue without cluttering classes.

The design also considers **connascence**. **Name** connascence appears in constants like `Condition.ALWAYS` and class names like `ListenAction`. **Type** connascence ties methods to structures like `List<Monologue>`. **Meaning** arises when methods assume consistent semantics across conditions. **Position** is seen in methods like `addMonologue(name, text, condition)`, where argument order matters. Recognizing these aids safe refactoring.

Overall, by combining inheritance for a clear NPC hierarchy with composition for flexible behaviors and conditions, the system achieves a modular, maintainable architecture. It is open to extension where needed, including new NPCs, monologue conditions, or actions, yet closed to modification of proven core logic. This balance ensures the framework can evolve alongside game requirements without sacrificing stability or readability.